Faculty of Information and Communication Technologies

Department of Computer Science

Computer Networks Course

Project I : Simulate the operation of the theoretical transport layer protocol **rdt 2.1**

Instructor: Dr. Ashraf Mohammed Alkhresheh

Student: Albara Khalid Ahmed Almseidiyen ,البراء خالد احمد المسيعديين

Objectives: The objectives of the RDT 2.1 (Reliable Data Transfer version 2.1) protocol are to ensure reliable communication over a potentially unreliable network. RDT 2.1 addresses the challenges of packet loss and corruption by introducing mechanisms such as checksums for error detection, sequence numbers to distinguish between new and retransmitted packets, and acknowledgment (ACK) and negative acknowledgment (NAK) packets to confirm successful reception or request retransmission of corrupted packets. The sender transmits a data packet and waits for an ACK to proceed or retransmits the packet if a NAK is received or if a timeout occurs, indicating potential packet loss. This version enhances the reliability of data transfer by handling both positive and negative acknowledgments explicitly and retransmitting packets when necessary to ensure that data is correctly and completely received by the receiver.

functionalities Description:
Sender (Client) Side Functionalities:

1. Data Retrieval (get_data_from_above):

The sender continuously waits to receive data from the application layer or the "above" layer. This data represents the payload that needs to be transmitted.

2. Packet Creation (make_pkt):

The sender creates a packet using the make_pkt function. This packet includes a sequence number (assumed to be 0 for simplicity), the data, and a checksum computed over the data to ensure integrity.

3. Sending Packet (udt_send):

The sender uses the udt_send function to send the packet over the unreliable data transfer (UDT) layer, simulating the actual transmission over a network.

4. Receiving Packet (rdt_rcv): After sending the packet, the sender waits to receive a packet (either an ACK or a NAK) from the receiver. The rdt_rcv function checks if the received packet is corrupted or not and whether it is an ACK or a NAK.

5. Handling ACK/NAK:

If an ACK is received and the packet is not corrupted, the sender prepares to send the next data.

If a NAK is received or if the packet is corrupted, the sender retransmits the original packet.

Receiver (Server) Side Functionalities:

The receiver side code is not provided, but typically it would have the following functionalities:

1. Receiving Data (udt_rcv):

The receiver waits to receive a packet from the sender.

2. **Packet Validation**

The receiver checks the received packet for corruption using the checksum.

If the packet is not corrupted, the receiver sends an ACK back to the sender.

If the packet is corrupted, the receiver sends a NAK to request retransmission.

3. Sending ACK/NAK (udt_send):

Depending on the validation result, the receiver sends an ACK to acknowledge the receipt of a valid packet or a NAK to request a retransmission.

4.Delivering Data to Application Layer:

After receiving a valid packet, the receiver extracts the data and delivers it to the application layer or the "above" layer.

How to run the codes:

1. **open two command prompts or windows powershells in the folder that have the client and server files.**
2. **Run the command (python albraa_khaled_server.py)in the first one, and (python albraa_khaled_client.py) in the other.**
3. **After running the two files the client will ask you to enter one character for send it to server.**
4. **After interring a character and press on (enter) the client will send that character to server, then the server will process it, detect errors (if any), and send ACK or NAK accordingly.**
5. **If the server sends NAK then the client will ask to inter the character again until the server send an ACK ,after that the client will ask you to inter the next character.**